Controlled Packet Capture and 802.11 Protocol Analysis Lab - EAP-PEAP Validation

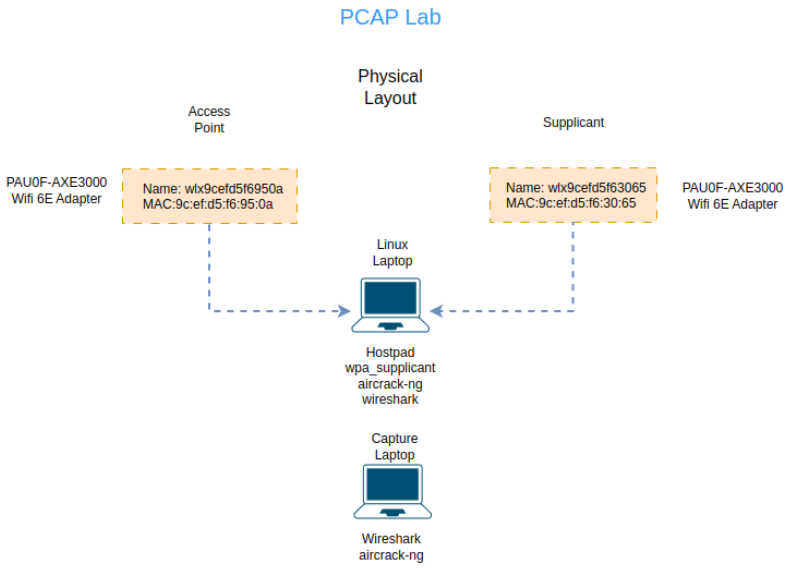
In this post, we will explore how to identify EAP authentication and its associated EAP type by analyzing network traffic captured from our PCAP Lab. Using packet analysis, we will identify the authentication method in use (such as EAP-PEAP) and understand how to recognize the different stages of the IEEE 802.1X authentication process.

Lab Setup

A picture is worth a thousand words as the saying goes! Following are diagrams depicting the Logical and Physical layout of our Lab environment. Going forward, we will reference the diagrams as required.

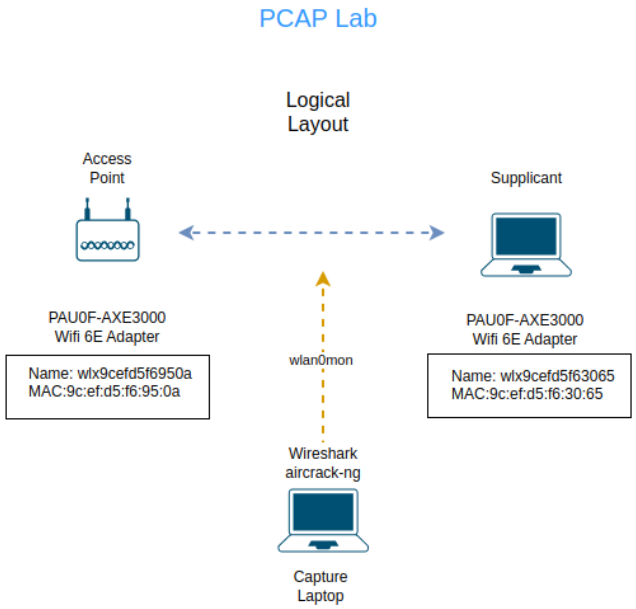
PCAP Lab Physical

Following highlights the physical components of the PCAP Lab. Please refer to the first post in this series for assistance with Lab setup if required.



PCAP Lab Logical

Logically, our lab has one supplicant (Client), one Access Point (AP), and a Laptop acting as a capture point. This laptop will follow our established process of creating a monitor interface via airmon-ng (wlan0mon) for utilization of packet capture. To minimize the length of the post, we will not be going into setup of Wireshark per post as this topic has been established previously. Instead, we will review previously captured traffic as to focus on analysis. Please note, during packet-capture, capture filters were applied to minimize the noise in our pcaps. Using capture filters can assist with minimizing noise in our pcaps, however, when possible, I suggest starting with no capture filters to expand the capture scope.



Access Point

Let's kick this off by initializing our access point. To do so, we will utilize the following hostapd configuration file. Notice the 'interface' and 'ssid'? Please change the interface to align with your interface naming.

hostapd_8021x.conf

```
interface=wlx9cefd5f6950a
driver=nl80211

ssid=PCAP-Lab
hw_mode=a
channel=36
country_code=US
beacon_int=100
ieee80211n=1

auth_algs=1
wpa=2
wpa_key_mgmt=WPA-EAP
rsn_pairwise=CCMP

ieee8021x=1

auth_server_addr=192.168.1.10
auth_server_port=1812
auth_server_shared_secret=Testlab12345

# Built-in EAP server
eap_server=1
eap_user_file=/etc/hostapd/hostapd.eap_user

# Server certificate for PEAP's TLS tunnel
ca_cert=/etc/hostapd/certs/ca.pem
server_cert=/etc/hostapd/certs/server.pem
private_key=/etc/hostapd/certs/server.key
private_key_passwd=

# Optional: PMF
ieee80211w=1
```

Initialize hostapd

```
sudo hostapd -i wlx9cefd5f6950a hostapd_8021x.conf
```

Result

```
wlx9cefd5f6950a: interface state UNINITIALIZED->ENABLED
wlx9cefd5f6950a: AP-ENABLED
```

Client (Supplicant)

Great! Now we have a functioning access-point for our lab. Let's work on the client. We will use the following configuration file when initializing a client connect to the access-point via the wpa_supplicant.

In the following wpa_supplicant file, the supplicant will initiate authentication to our Lab AP and utilize 802.1X/EAP-PEAP. During this process, the supplicant (Client), will attempt to validate the (Authentication Server/RADIUS) server certificate. If the validation is successful, the (Outer Tunnel/Phase1) will be established. The subsequent steps in this process will then move to (Inner Tunnel/Phase2) where client credentials will be passed. The client credentials are secured by the Outer tunnel. It is important to note that Phase2 authentication can utilize multiple options. A common scenario would be to utilize MSCHAPv2.

Please notice the line that is "uncommented" and starting with "ca_cert". This is the wpa_supplicant option that instructs the client to "validate" the (Authentication Server/RADIUS) server cert. In our lab, and in this config file, we are not using a CA Cert that can validate the (hostapd) server cert. To that end, the outer tunnel/Phase1 will fail.

!! This is exactly what we want to see !!

wpa_8021x_eap_peap_validate_server_cert.conf

```
#ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1

network={
    ssid="PCAP-Lab"
    key_mgmt=WPA-EAP
    eap=PEAP
    identity="MyName"
    anonymous_identity="MyName"
    password="mypassword"
    phase2="auth=MSCHAPV2"
    # Critical for validation:
    ca_cert="/usr/share/ca-certificates/mozilla/SSL.com_TLS_RSA_Root_CA_2022.crt"
    # Optional tighter checks:
    # subject_match="/CN=expected-radius.example.com"
    # domain_suffix_match="example.com"
    # phase1="peapver=0 peaplabel=0"
}
```

Initiate client connection

Prior to initiating a connection attempt to our access-point, we have started Wireshark on the capture laptop, utilizing our monitor interface wlan0mon. Perfect, let's attempt a connection.

```
sudo wpa_supplicant -i wlx9cefd5f63065 -c
wpa_8021x_eap_peap_validate_server_cert.conf
```

Client connecting

```
wlx9cefd5f63065: CTRL-EVENT-EAP-PEER-ALT depth=0 DNS:radius.example.local
TLS: Certificate verification failed, error 19 (self-signed certificate in
certificate chain) depth 1 for '/C=US/O=MyLab/CN=MyLab Root CA'
wlx9cefd5f63065: CTRL-EVENT-EAP-TLS-CERT-ERROR reason=1 depth=1
subject='/C=US/O=MyLab/CN=MyLab Root CA' err='self-signed certificate in
certificate chain'
SSL: SSL3 alert: write (local SSL3 detected an error):fatal:unknown CA
OpenSSL: openssl_handshake - SSL_connect error:0A000086:SSL
routines::certificate verify failed
wlx9cefd5f63065: CTRL-EVENT-EAP-FAILURE EAP authentication failed
wlx9cefd5f63065: CTRL-EVENT-DISCONNECTED bssid=9c:ef:d5:f6:95:0a reason=23
wlx9cefd5f63065: CTRL-EVENT-SSID-TEMP-DISABLED id=0 ssid="PCAP-Lab"
auth_failures=1 duration=10 reason=AUTH_FAILED
^Cwlx9cefd5f63065: CTRL-EVENT-DSCP-POLICY clear_all
wlx9cefd5f63065: CTRL-EVENT-DSCP-POLICY clear_all
nl80211: deinit ifname=wlx9cefd5f63065 disabled_11b_rates=0
wlx9cefd5f63065: CTRL-EVENT-TERMINATING
```

Great! If we are watching our terminal that we used initialized hostapd, we will see a message indicating success or failure!

hostapd - access-point

```
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: authenticated
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: associated (aid 1)
wlx9cefd5f6950a: CTRL-EVENT-EAP-STARTED 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: CTRL-EVENT-EAP-PROPOSED-METHOD vendor=0 method=1
wlx9cefd5f6950a: CTRL-EVENT-EAP-PROPOSED-METHOD vendor=0 method=25
SSL: SSL3 alert: read (remote end reported an error):fatal:unknown CA
OpenSSL: openssl_handshake - SSL_connect error:0A000418:SSL routines::tlsv1
alert unknown ca
wlx9cefd5f6950a: CTRL-EVENT-EAP-FAILURE 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.1X: authentication failed -
EAP type: 0 (unknown)
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.1X: Supplicant used different
EAP type: 25 (PEAP)
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: deauthenticated due to
local deauth request
```

Success! Well, more accurately, Success for Failing! We know that our supplicant is attempting to validate the server cert and that we "intentionally" configured a CA Cert that will not successfully validate the server cert. The purpose is to verify the EAP Type and Failure, more to the point, the client is properly validating the server cert and failing, via Packet capture. We can clearly do that now with this capture.

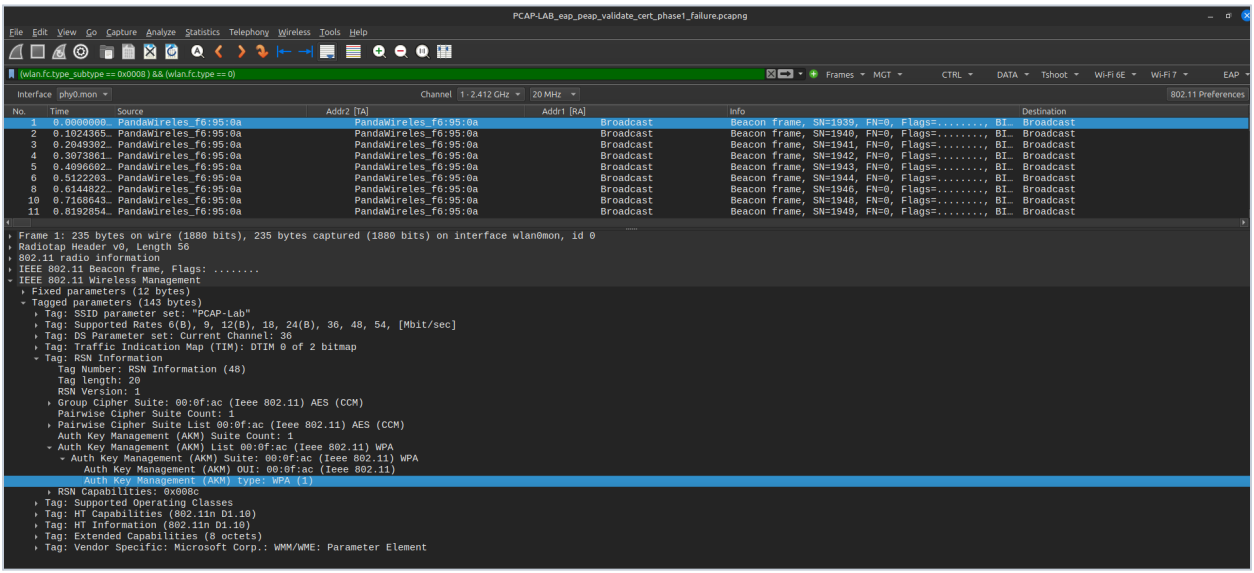
802.1X Analysis

Now that we established that our client is attempting to authenticate with our access-point via EAP-PEAP, let's identify the same information via a packet capture.

Identify 802.1X

PCAP-LAB_eap_peap_validate_cert_phase1_failure.pcapng

Recall from our previous posts that Beacons (Management Frame) carry a wealth of information? Let's have a look and see if we can Identify what type of AKM (Authentication and Key Management) is in use.



The AKM information we are concerned with is carried within "Tagged parameters" which reside in the frame body. These parameters consist of IE (Information Elements). IEs carry additional information.

Taking a closer look, drilling down into "Tag: RSN Information" / "Auth Key Management" we find our "AKM Key Management Suite". We can identify via the IEEE standard OUI of "00:0f:ac" and the AKM "type" of (1) that 802.1X is used. Following is a list of common AKM suite identifiers. Notice the (1) for 802.1X?

Authentication and Key Management (AKM) suites

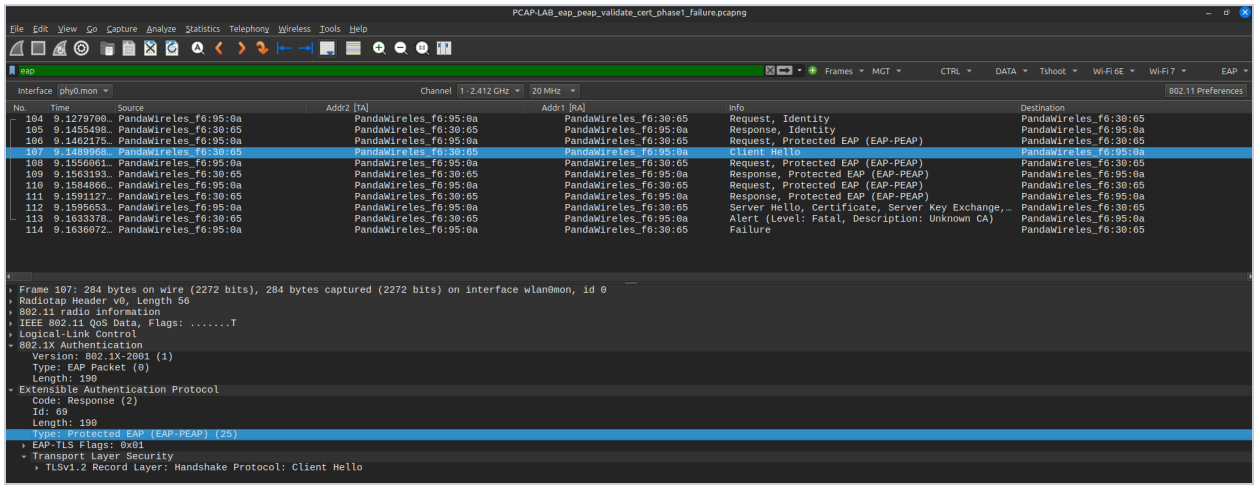
Value	Meaning
00:0F:AC:01	802.1X / EAP
00:0F:AC:02	PSK
00:0F:AC:03	FT over 802.1X
00:0F:AC:04	FT using PSK
00:0F:AC:08	SAE

Great! We know 802.1X is used, but we don't know which type! Let's see how we can identify the type of EAP.

Identify 802.1X EAP type

Using the same packet capture, let's look at how we can identify the type of EAP in use. First, let's clear up some understanding. The EAP type is negotiated between the "Client" and "Authentication Server / RADIUS". This negotiation happens after Association. This is where we will have to look.

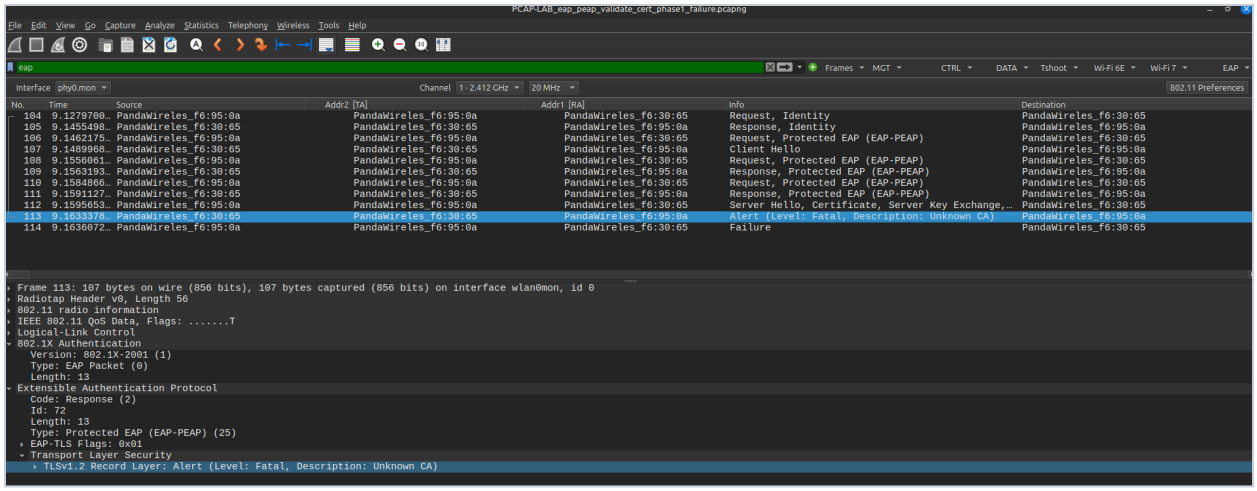
Let's apply a display filter of "EAP"



The server kicks things off by proposing EAP-PEAP in the packet just before the 'Client Hello' ("Request, Protected EAP (EAP-PEAP)"). The client accepts by answering with its 'Client Hello'. Looking under "802.1X Authentication", "Extensible Authentication Protocol", "Type", we can see the client is now using EAP-PEAP! Looking at the pcap, info column, we can quickly identify the same information.

Identify Server Cert Validation

Great! We now know how to identify that 802.1X / EAP is in use and how to identify the type of EAP. In our case, PEAP. If we recall, we also set out to explore what a 'Validation Failure' looks like, more specifically, if the client is configured to validate the server cert. To do so, we configured our client (wpa_supplicant) to validate the server certificate with a "bogus/not the CA cert that signed the server certificate", to capture the behavior of the client. Would the client establish the "outer tunnel/Phase1" or would it "fail" due to invalid cert. Let's have a look at the pcap.



We can verify by our pcap that the client indeed validated the server cert, and failed! Fantastic! This is exactly what we want our client to do! "If I don't trust you, I am not going to talk to you!"

Have a look at "802.1X Authentication", "Extensible Authentication Protocol", "Transport Layer Security", "TLSv1.2 Record Layer".

We can easily see the same from the "Info column". "Unknown CA".

NOT validating the Server Cert

Let's now have a look at a client that is configured to "NOT" validate the Authentication Server Cert.

I can not stress this enough!! This is bad practice to allow a client to bypass server cert validation. Doing so presents the possibility to EvilTwin attacks. EvilTwin is when impersonation of a known SSID is accomplished. The client does not validate server cert, accepting all certs as valid. The client then established a connection with the "FAKE" SSID. The operator of the "FAKE" SSID can now capture the MSCHAPv2 challenge/response and crack it offline to recover the user's password.

Client configuration file

Let's change our 'wpa_supplicant' config file and "comment out" the line for the "ca_cert". Taking this step will signal to the client to "NOT Validate" server certificates. The client will then treat all certificates as valid, build the "outer tunnel/Phase1", and continue to Inner Tunnel authentication.

```
sudo wpa_supplicant -i wlx9cefd5f63065 -c
wpa_8021x_eap_peap_no_validate_server_cert.conf
```

Notice the line "ca_cert" is commented!

```
#ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1

network={
    ssid="PCAP-Lab"
    key_mgmt=WPA-EAP
    eap=PEAP
    identity="MyName"
    anonymous_identity="MyName"
    password="mypassword"
    phase2="auth=MSCHAPV2"
    # Critical for validation:
    # ca_cert="/usr/share/ca-certificates/mozilla/SSL.com_TLS_RSA_Root_CA_2022.crt"
    # Optional tighter checks:
    # subject_match="/CN=expected-radius.example.com"
    # domain_suffix_match="example.com"
    # phase1="peapver=0 peaplabel=0"
}
```

Initiate client connection

```
sudo wpa_supplicant -i wlx9cefd5f63065 -c
wpa_8021x_eap_peap_no_validate_server_cert.conf
```

Client connecting

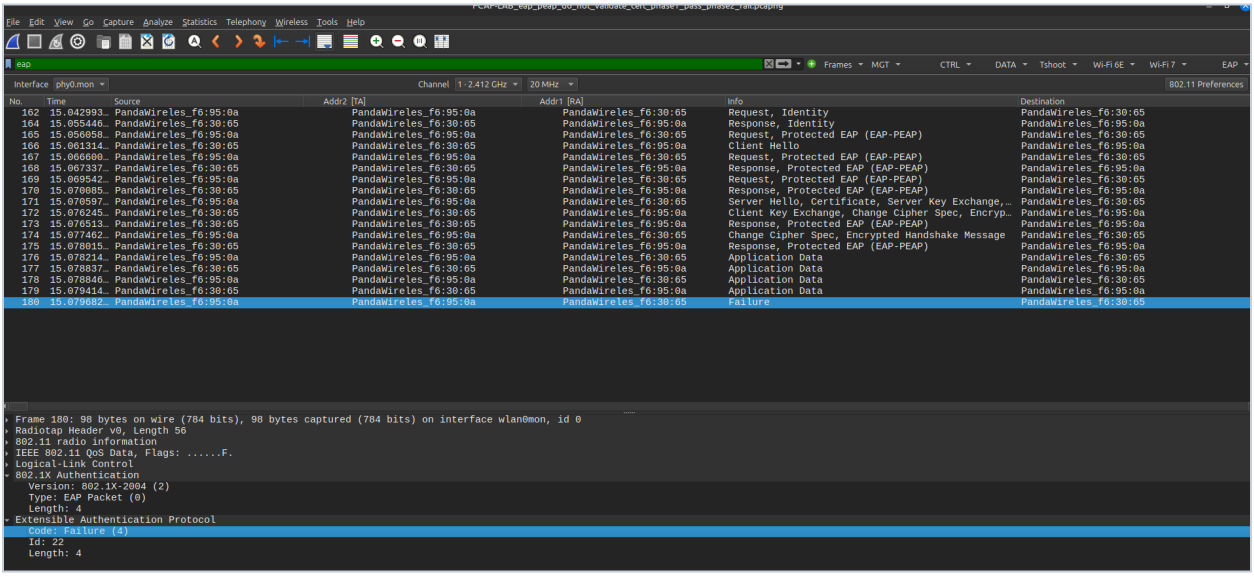
```
wlx9cefd5f63065: SME: Trying to authenticate with 9c:ef:d5:f6:95:0a
(SSID='PCAP-Lab' freq=5180 MHz)
wlx9cefd5f63065: Trying to associate with 9c:ef:d5:f6:95:0a (SSID='PCAP-Lab'
freq=5180 MHz)
wlx9cefd5f63065: Associated with 9c:ef:d5:f6:95:0a
wlx9cefd5f63065: CTRL-EVENT-EAP-STARTED EAP authentication started
wlx9cefd5f63065: CTRL-EVENT-SUBNET-STATUS-UPDATE status=0
wlx9cefd5f63065: CTRL-EVENT-EAP-PROPOSED-METHOD vendor=0 method=25
wlx9cefd5f63065: CTRL-EVENT-EAP-METHOD EAP vendor 0 method 25 (PEAP) selected
wlx9cefd5f63065: CTRL-EVENT-EAP-PEER-CERT depth=1
subject='/C=US/O=MyLab/CN=MyLab Root CA'
hash=5850b5289c75dbc85f43d15ad32d1e4b4c7df434f36c822c7c5ed24b4be7934e
wlx9cefd5f63065: CTRL-EVENT-EAP-PEER-CERT depth=1
subject='/C=US/O=MyLab/CN=MyLab Root CA'
hash=5850b5289c75dbc85f43d15ad32d1e4b4c7df434f36c822c7c5ed24b4be7934e
wlx9cefd5f63065: CTRL-EVENT-EAP-PEER-CERT depth=0
subject='/C=US/O=MyLab/CN=radius.example.local'
hash=2c2116721914c306a38a8e8d02560a10eaeed5f27cf83a35149ac8c8ab5cea70
wlx9cefd5f63065: CTRL-EVENT-EAP-PEER-ALT depth=0 DNS:radius.example.local
wlx9cefd5f63065: CTRL-EVENT-EAP-FAILURE EAP authentication failed
wlx9cefd5f63065: CTRL-EVENT-DISCONNECTED bssid=9c:ef:d5:f6:95:0a reason=23
wlx9cefd5f63065: CTRL-EVENT-SSID-TEMP-DISABLED id=0 ssid="PCAP-Lab"
auth_failures=1 duration=10 reason=AUTH_FAILED
^Cwlx9cefd5f63065: CTRL-EVENT-DSCP-POLICY clear_all
wlx9cefd5f63065: CTRL-EVENT-DSCP-POLICY clear_all
nl80211: deinit ifname=wlx9cefd5f63065 disabled_11b_rates=0
wlx9cefd5f63065: CTRL-EVENT-TERMINATING
```

hostapd - access-point

```
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: authenticated
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: associated (aid 1)
wlx9cefd5f6950a: CTRL-EVENT-EAP-STARTED 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: CTRL-EVENT-EAP-PROPOSED-METHOD vendor=0 method=1
wlx9cefd5f6950a: CTRL-EVENT-EAP-PROPOSED-METHOD vendor=0 method=25
wlx9cefd5f6950a: CTRL-EVENT-EAP-FAILURE 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.1X: authentication failed -
EAP type: 0 (unknown)
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.1X: Supplicant used different
EAP type: 25 (PEAP)
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: deauthenticated due to
local deauth request
```

Packet Capture verification

Notice the additional "Key Exchange" messages. The client was configured to "NOT Validate the server certificate". This means the client will "trust" any certificate. This occurred causing the process to continue and the exchange of keying material. The client then moved to the "Inner Tunnel/Phase2" and passed credentials. At that point, the client failed as we did not configure the user's credentials on the authentication server.



We can clearly see the importance of aligning client side configuration to validate server certificate.

Certificate Verification via Wireshark

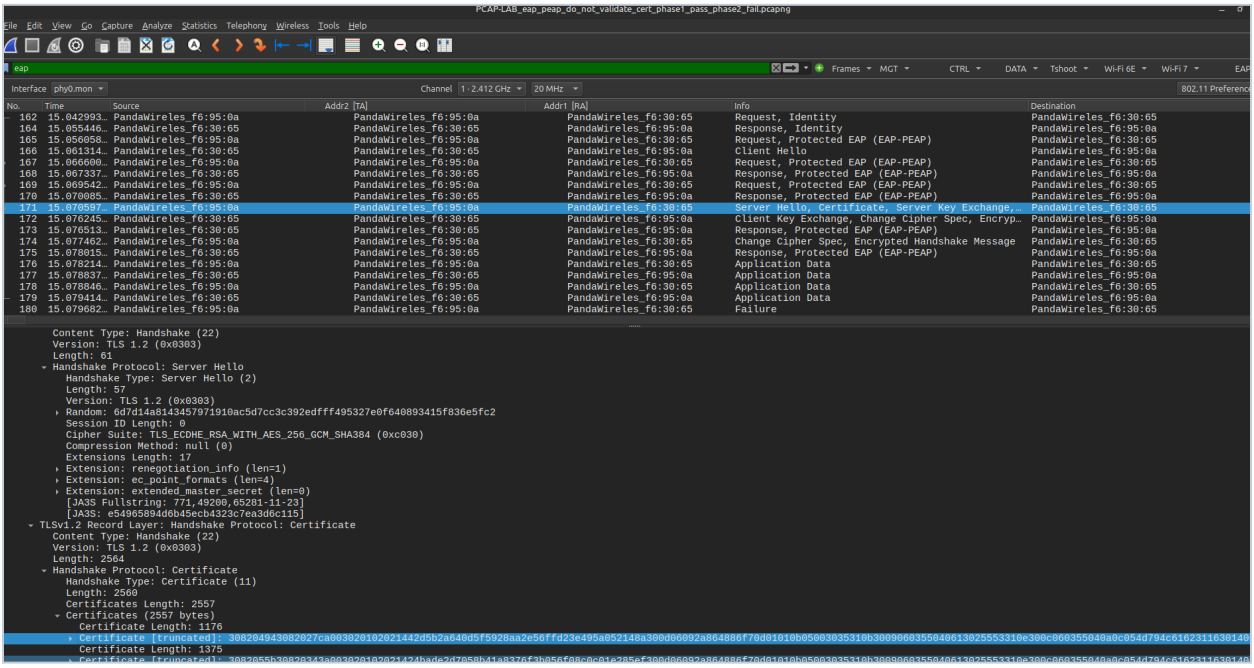
As a side note, a common call I receive is that 802.1X clients suddenly can't connect. More often than not, it comes down to a certificate issue, such as an expired or replaced RADIUS server certificate. Certificates tend to be a struggle for a lot of folks, so I wanted to show how to verify them, more specifically, how to export a certificate straight from a packet capture.

Identify the Certificate

To highlight this process, we will use our EAP-PEAP pcap.

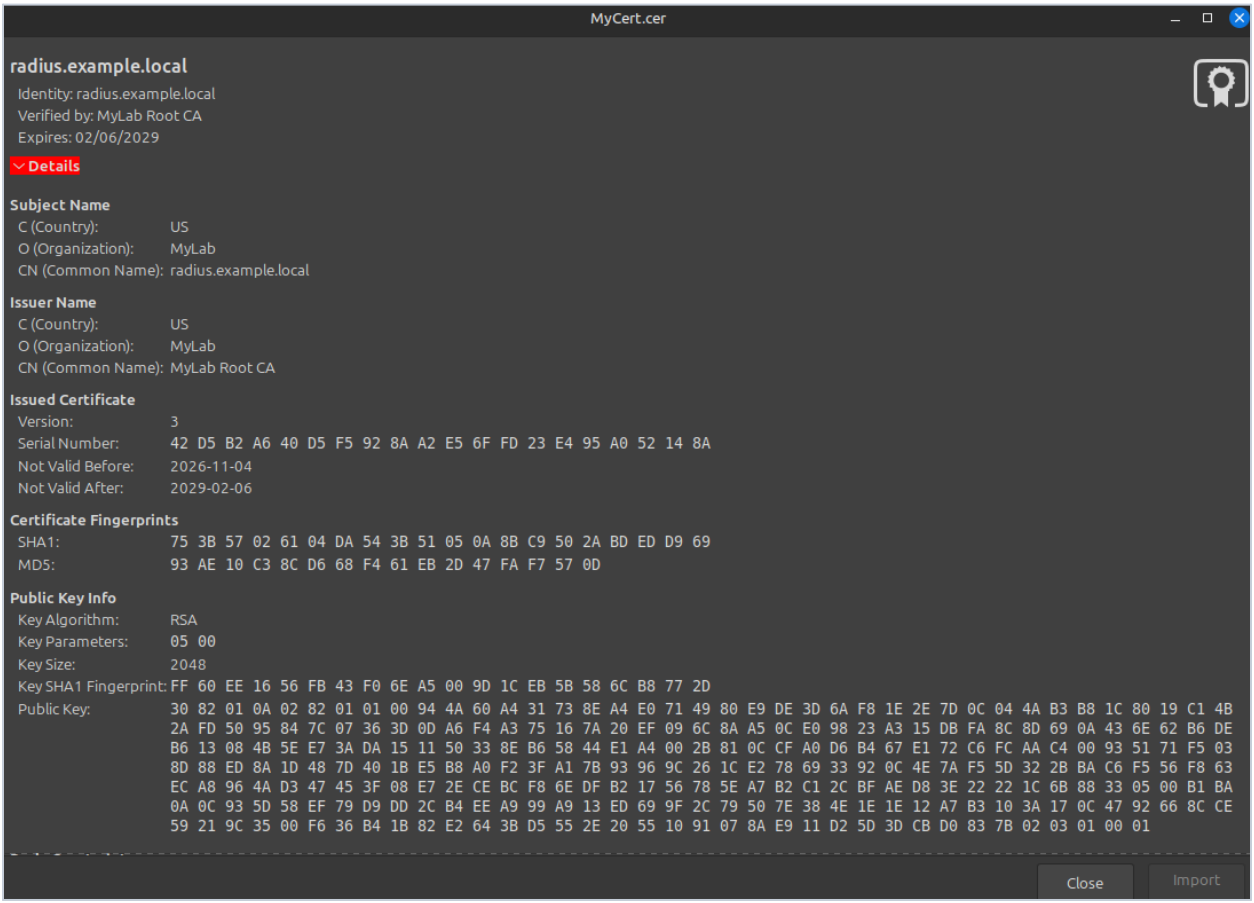
Let's first look in the "Info column" and identify the "Server Hello, Certificate, Server Key Exchange". Really, you could do the same in any other frame that has the certificate info.

Next, let's drill down into "Extensible Authentication Protocol", "Transport Layer Security", "TLSv1.2 Record Layer", "Handshake Protocol", "Certificates". Now, we can highlight the "Certificate" we want to view, "right click", choose "Export Packet bytes". Save the output. Name the file something useful. Make sure you give the file extension as ".cer" or ".crt". Next, you can open the file and view the certificate.



View Exported Certificate

Let's view our exported certificate!



Conclusion

In this post we explored how to setup our Lab to utilize 802.1X AKM. We then identified, via packet capture analysis, how we can identify that 802.1X/EAP was in use. Next, we moved forward with identification of the type of EAP. In our case, EAP-PEAP. Once established, we took a look at how to identify client configuration of "validate Server certificate" and what that looks like from a packet capture view point. We then configured our client to "Not Validate Server certificates" and reviewed the packet capture to identify the view and behavior. Lastly, we demonstrated how to export certificates from packet captures to assist with identifying details of those certificates.

Most importantly, we identified the security impact of client side decisions, and why it is important to control those decisions.

In our next lab, we will explore the Retry field, how to quickly identify it using a Wireshark display filter, and the impact of high retry rates. We will also introduce pseudo headers and look at how we can use this information.